

FASTLANE · SECURITY REVIEW

FastLane Consultation Review.

A security review of Consultation Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
FastLane	Consultation Review	Guardian	September 11 – September 15, 2025	September 24, 2025

§ 00

Contents.

27 pages — overview, confidence ranking, scope, 13 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 13 findings	P. 09
§ 05	Disclaimer	P. 26
§ 06	About Guardian	P. 27

§ 01

Overview.

Guardian reviewed Consultation Review across a single review round. The final report includes 1 Critical, 3 High, 2 Medium, 2 Low, 5 Informational findings.

AUDIT FIRM

Guardian

CLIENT

FastLane

PROJECT

Consultation Review

PREPARED BY

Guardian

AUDIT WINDOW

September 11 – September 15, 2025

FINAL REPORT

September 24, 2025

REPOSITORY

FastLane-Labs/fastlane-contracts-internal

METHODOLOGY

Smart contract · Manual · Main Review

1.1 Confidence Ranking

3 / 5

**Moderate
Confidence**

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	1	0	1	0	0
● High	3	0	3	0	0
● Medium	2	0	2	0	0
● Low	2	0	2	0	0
● Informational	5	0	5	0	0
All severities	13	0	13	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Consultation Review in `FastLane-Labs/fastlane-contracts-internal`. The scope includes 1,777 source lines across 18 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/shmonad/AtomicUnstakePool.sol	91	175
src/shmonad/CapitalAllocator.sol	350	688
src/shmonad/Constants.sol	13	16
src/shmonad/DirectDelegation.sol	81	195
src/shmonad/Errors.sol	47	64
src/shmonad/Events.sol	58	67
src/shmonad/FLERC20.sol	155	351
src/shmonad/FLERC4626.sol	170	373
src/shmonad/Holds.sol	45	134
src/shmonad/Policies.sol	310	763
src/shmonad/PolicyERC20Wrapper.sol	14	23
src/shmonad/ShMonad.sol	111	280
src/shmonad/Storage.sol	77	128
src/shmonad/Types.sol	81	122
src/shmonad/libraries/BitmapSetLib.sol	43	76
src/shmonad/libraries/FeeLib.sol	74	152
src/shmonad/libraries/HoldsLib.sol	40	58
src/shmonad/libraries/ValidatorLib.sol	17	30

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Yield Realization is Sandwichable	MEV	CRITICAL	ACKNOWLEDGED	p. 9
H-01	Double Counting Normal Yield in Stake Totals	Logical Error	HIGH	ACKNOWLEDGED	p. 10
H-02	Incorrect Unstake Accounting in Manual Functions	Logical Error	HIGH	ACKNOWLEDGED	p. 11
H-03	User Controllable Staking Bias	Rewards	HIGH	ACKNOWLEDGED	p. 12
M-01	Solady's Hardcoded WETH9 and Monad Incompatibility	Configuration	MEDIUM	ACKNOWLEDGED	p. 13
M-02	Epoch Misalignment Creates Sandwichable Yield Realization	Rewards	MEDIUM	ACKNOWLEDGED	p. 14
L-01	Permissonless Manual Functions	Best Practices	LOW	ACKNOWLEDGED	p. 15
L-02	Liquidity Seeding During Initialization Creates Distorted Share Pricing	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 16
I-01	Current Architecture Doesn't Support Slashing Case	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-02	Architecture Suggestions	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 18
I-03	On Fastlane's unique totalSupply (including isolated shares)	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-04	ERC4626 Compliance	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-05	Validators Handling	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 24

CRITICAL **C-01**

ACKNOWLEDGED

Yield Realization is Sandwichable

CATEGORY	MEV
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

All shMonad stake and unstake flows rely on `realTotalAssets`, which factors in both standard staking APR and boosted yield (e.g., MEV rewards). Boosted yield is escrowed until realized, at which point a stepwise jump occurs. Because `realizeYield()` is permissionless, any actor can trigger this jump at a favorable moment, enabling sandwich attacks that dilute other holders.

The reason for this being critical is that this would allow anyone to sandwich all unrealized yield of all validators all at once at their wish.

Atlas-triggered realizations are also frontrunnable, exposing systemic risk. Considering shMonad is a liquid staking protocol, this vulnerability affects core functionality.

In case of Fastlane, there are 2 types of yield, one is from monad staking contract, this is only updated at end of epoch on source, hence makes sense to realize and update ratios at start of each epoch.

However boosted yields can happen anytime during the epoch, and ideally users should not be able to realise those by staking post boost action.

— RECOMMENDATION

Smooth out yield distribution using a pending-yield mechanism that amortizes over time (see example implementation: [GuardianAudits gist]

(<https://gist.github.com/GuardianAudits/c834a37f225cb5c94f8eace49aa1e1c4>)).

As a minimal interim step, restrict `boostYield` with frontrun-resistant logic and perform yield realization within boost calls.

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

Double Counting Normal Yield in Stake Totals

CATEGORY	Logical Error
LOCATION	https://github.com/FastLane-Labs/fastlane-contracts-internal/blob/88a24b4ee1f4526aa116133f080501a250376c47/src/shmonad/DirectDelegation.sol#L180
REVIEW	Main Review

DESCRIPTION

In the current yield flow:

- `v.iso` and `v.floating` are incremented by both normal yield and boosted yield.
- Boosted yield is staked (`stakeToValidator`), but `**normal yield remains in address(this).balance**.`
- Despite this, `stakeTotals.floating` is incremented for the full amount, effectively double-counting normal yield in `realTotalAssets`.

RECOMMENDATION

Consider using `_getValidatorBalanceAndCompoundRewards` instead of `getValidatorBalance`. Ensure compounding occurs before querying delegator info:

1. Fetch validator info.
2. If rewards > 0, compound them.
3. Fetch updated info post-compound.

RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Incorrect Unstake Accounting in Manual Functions

CATEGORY	Logical Error
LOCATION	https://github.com/FastLane-Labs/fastlane-contracts-internal/blob/88a24b4ee1f4526aa116133f080501a250376c47/src/shmonad/StakingIntegration.sol#L253
REVIEW	Main Review

DESCRIPTION

In `manuallyInitiateUnstakeFromValidators`, `_unstakeTotals.requestedMon` is reduced only by `_amountUnstaked`. However, `initiateUnstakeFromValidators` can also draw from pool liquidity, which is not deducted. This causes divergence from `startEpoch` accounting.

RECOMMENDATION

Consider updating `manuallyInitiateUnstakeFromValidators` so that `_unstakeTotals.requestedMon` is reduced by both `_amountUnstaked` and any liquidity drawn.

RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

User Controllable Staking Bias

CATEGORY Rewards

LOCATION <https://github.com/FastLane-Labs/fastlane-contracts-internal/blob/88a24b4ee1f4526aa116133f080501a250376c47/src/shmonad/StakingIntegration.sol#L488>

REVIEW Main Review

DESCRIPTION

`_initiateUnstakeFromValidators` unstakes on a pro-rata basis across all validators. However, staking deposits stop early if gas is insufficient:

```
if (amount < MIN_VALIDATOR_DEPOSIT) break;  
if (gasleft() < STAKE_ITERATION_MIN_GAS) break;
```

This allows users to provide just enough gas to stake only into a subset of validators (e.g., v1, v2), while unstake still pulls pro-rata from all (v1-v4). As a result, some validators' yields may remain unrealized and manipulable.

RECOMMENDATION

Consider enforcing yield realization across all validators before any `convertToAssets` or `convertToShares` calculation.

Note that if C-1 is resolved with realize yield on each boosted call, this won't remain issue anymore.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Solady's Hardcoded WETH9 and Monad Incompatibility

CATEGORY Configuration**LOCATION** <https://github.com/Vectorized/solady/blob/208e4f31cfae26e4983eb95c3488a14fdc497ad7/src/Utils/SafeTransferLib.sol#L59>**REVIEW** Main Review

— DESCRIPTION

The Solady library hardcodes WETH9 for ETH safe transfers. Since Fastlane operates on Monad (with MON as native currency), this behavior will not function correctly. Additionally, Solady makes several opcode-level assumptions that may not hold true on Monad, and such inconsistencies might not be caught at compilation time. Given Monad's relatively new environment, Solady remains untested in production.

— RECOMMENDATION

Adopt OpenZeppelin (OZ) contracts instead of Solady until the latter has been fully tested on Monad. OZ offers safer, more conservative defaults and avoids risky assumptions.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Epoch Misalignment Creates Sandwichable Yield Realization

CATEGORY Rewards

LOCATION <https://github.com/FastLane-Labs/fastlane-contracts-internal/blob/88a24b4ee1f4526aa116133f080501a250376c47/src/shmonad/StakingIntegration.sol#L144>

REVIEW Main Review

DESCRIPTION

Fastlane's epoch clock is expected to run in sync with Monad's epoch clock. `startEpoch` batches yield realization across all validators. If there is a delay in calling `startEpoch`, inconsistencies arise:

- Users can deposit during a new Monad epoch but still accrue yield from the previous one, since Fastlane has not updated yet.
- This makes `startEpoch` sandwichable: attackers can position transactions around epoch transitions to capture guaranteed APR.

Because standard Monad APR is guaranteed, an attacker can always place their transaction immediately after a Monad epoch update but before Fastlane's update, capturing unearned yield.

RECOMMENDATION

- Keep `startEpoch` strictly synchronized with Monad's epoch boundaries.
- Implement yield smoothing logic here as well, ensuring yield accrual is continuous rather than stepwise.
- As a minimal interim measure, use a frontrun-resistant executor service to call `startEpoch`. However, this is not a full solution, since attackers can still time deposits relative to Monad's guaranteed epoch updates.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Permissionless Manual Functions

CATEGORY	Best Practices
LOCATION	StakingIntegration.sol
REVIEW	Main Review

DESCRIPTION

All manual functions are currently permissionless, exposing unnecessary attack surface without clear benefits.

RECOMMENDATION

Access Control: Consider restricting manual functions to role-based calls using OpenZeppelin's `onlyRoleOrOpen` modifier in initial stages, allowing later migration to permissionless operation if desired.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Liquidity Seeding During Initialization Creates Distorted Share Pricing

CATEGORY Unexpected Behavior

LOCATION <https://github.com/FastLane-Labs/fastlane-contracts-internal/blob/88a24b4ee1f4526aa116133f080501a250376c47/src/shmonad/AtomicUnstakePool.sol#L43>

REVIEW Main Review

DESCRIPTION

`__AtomicUnstakePool_init` seeds the pool liquidity during contract initialization:

```
PoolLiquidity memory _liquidity = PoolLiquidity({
currentLiq: address(this).balance.toUint128(),
targetLiqPercentage: BPS_SCALE // 100% for initial tests
});
s_poolLiquidity = _liquidity;
```

If the proxy is deployed, funded with MON, and then initialized:

- Example: pool is seeded with 1000 MON in `initialize()`.
- At this point, `_realTotalSupply()` = 0, but due to offset it is treated as 1.
- `_totalAssets` = 1000 MON.
- Result: 1 wei of share = 1000 MON, permanently distorting vault share pricing.

RECOMMENDATION

- Do not seed pool liquidity inside `initialize()`.
- Instead, deploy the contract, allow deposits to mint shares, and then add liquidity separately. This ensures that initial share pricing reflects a fair and accurate ratio between deposited MON and issued shares.

Fastlane Team confirmed that this feature was kept for supporting their testnet deployments and not meant for main deployment, hence severity reduced to low.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Current Architecture Doesn't Support Slashing Case

CATEGORY Best Practices
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Fastlane's contracts throughout assume vault balances always increase monotonically. However, if Monad introduces validator slashing, this assumption breaks.

— RECOMMENDATION

Beware of this constraint

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Architecture Suggestions

CATEGORY	Best Practices
LOCATION	—
REVIEW	Main Review

DESCRIPTION

Suggestion 1 : Isolated vs Total via Validator Vaults + Meta Vault

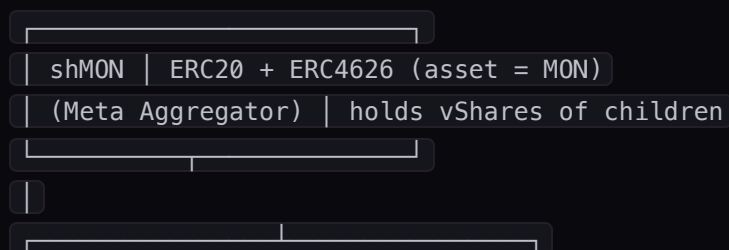
Proposed design:

- Each validator has its own ERC-4626 vault, where users can deposit/withdraw MON directly and receive vShares (non-transferable for now; could be transferable later).
- On top of these, a meta-vault (shMON) acts as an aggregator. Users can deposit here to mint shMON, and the meta-vault distributes stake across validator vaults and holds the resulting vShares.

Key points:

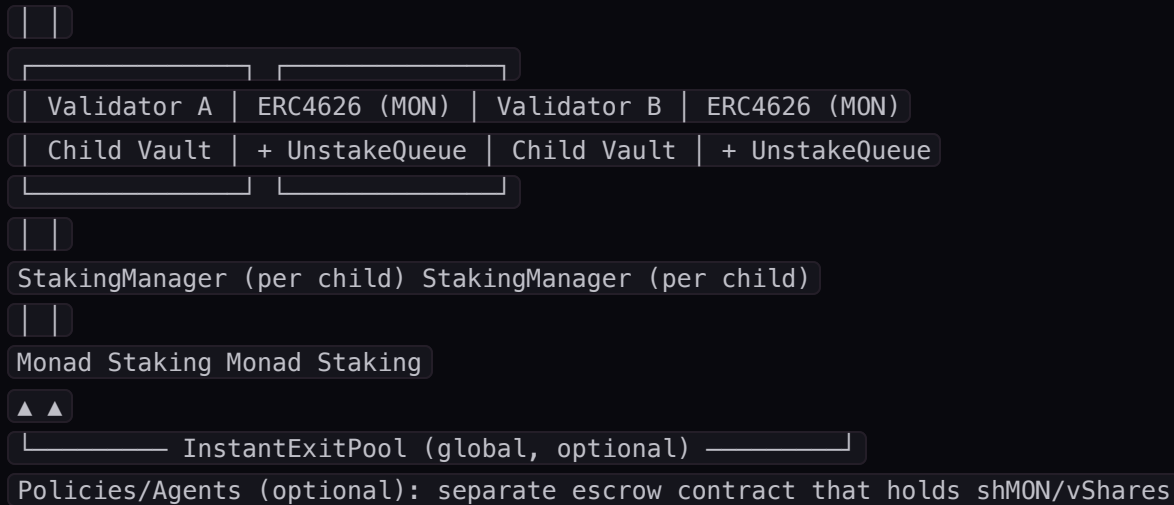
- Separation of concerns: accounting for direct validator deposits and meta-vault deposits stays isolated.
- TVL tracking: shMON's supply won't automatically reflect isolated validator stakes, but we can provide an aggregate view via (a) a wrapper function in the meta-vault, or (b) a peripheral contract.
- We can add a single entry point contract + factory to manage validator vaults. All calls route through the entry point, which also serves as a unified TVL/accounting view.
- Boosted yield: can be directed to a specific validator vault if needed, or prorated across vaults for global boosts.
- Upgradability: implement validator vaults as a beacon proxy, so upgrades apply to all vaults at once.
- Business needs: BD/sales teams can still get a single aggregated TVL number .

Architecture Diagram (View In Full Screen)



I-02 Architecture Suggestions

CONTINUED



Gas cost notes (proposed design vs current):

- User-facing actions (deposit/withdraw): No material change. Deposits route into the instant-unstake pool and staking is batched by the owner; withdrawals from the pool are independent of the child-vault structure.
- Yield realization: Same cost as today. For a given validator, the vault touches the same set of storage slots in both designs.
- What does change: We introduce a few additional external calls (meta-vault $\leftrightarrow$ child vaults, and child $\leftrightarrow$ staking precompile). These calls are independent and add overhead, which I think is acceptable given the separation of concerns.

Net: normal UX paths remain effectively the same in gas terms; the main delta is extra EXTCALLs from modularization.

Considering monad being chain with higher TPS, gas cost expense shouldnt be that much.

Suggestion 2: Reducing Inheritance and adding composition.Core Vault

`ShMonadVault.sol` (ERC20 + ERC4626 + Holds + StakingIntegration + Delegation)

- Mints/burns shMON.
- Tracks balances and validator delegations.
- Does not embed allocation or pool math.

Allocator + Pool (combined module)

`AllocatorUnstakePool.sol`

- Decides how to allocate stake across validators.
- Manages the atomic unstake pool for instant exits.
- Handles:

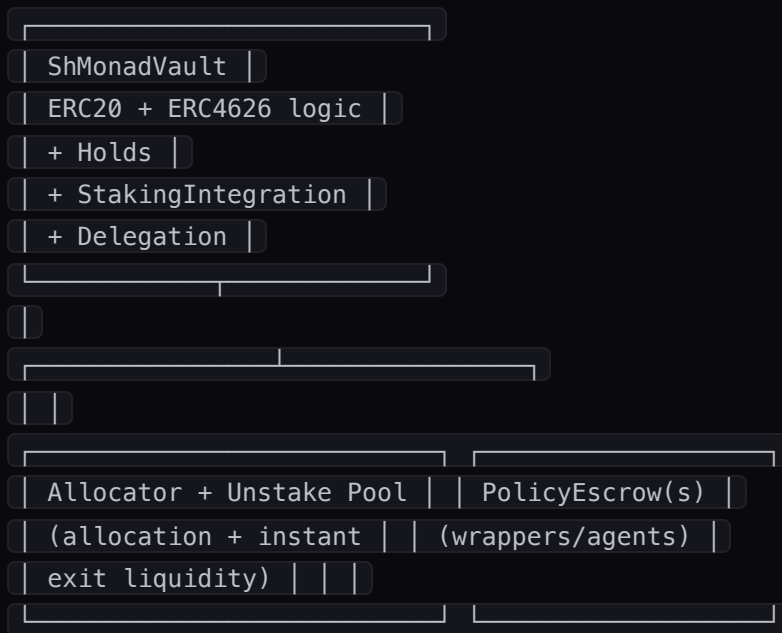
I-02 Architecture Suggestions

CONTINUED

- Pool liquidity (`cash` , fee curve).
- Splitting requests: what's satisfied via unstake vs via pool.
- Reporting deltas back to the Vault.
- Policies

Independent escrow contracts that just hold shMON/vShares. No internal hooks needed in the Vault.

Diagram

**— RECOMMENDATION**

Not recorded.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

On Fastlane's unique totalSupply (including isolated shares)

CATEGORY Best Practices
LOCATION —
REVIEW Main Review

DESCRIPTION

1. **Reliance on `totalSupply` in DeFi**

In practice, most DeFi protocols don't use `totalSupply` as an input to economic logic. They rely on balances, allowances, and exchange rates. So including "isolated shares" (that aren't actually minted) in `totalSupply` won't break core integrations.

2. Off-chain tooling impact

Where it can cause friction is with off-chain consumers (indexers, dashboards, subgraphs). These systems usually assume ERC20 `totalSupply` = sum of all minted balances. If Fastlane diverges from this assumption, indexers will need special-case logic to reconcile, which adds maintenance overhead and could confuse integrators.

If the goal is to surface "aggregate TVL" that includes isolated positions, it may be better to:

- Keep `ERC20.totalSupply()` standard (minted balances only),
- Add a separate view function (e.g., `totalSupplyWithIsolation()` or `aggregateSupply()`) that reports the augmented number.

RECOMMENDATION

Not recorded.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

ERC4626 Compliance

CATEGORY Informational
 LOCATION —
 REVIEW Main Review

DESCRIPTION

| Function / Feature | Must satisfy / behavior | Pass / No Pass |

| --- | --- | --- |

| ERC-20 base (transfer, approve, balanceOf, totalSupply) | Vault shares must implement standard ERC-20 behaviors (transfers, allowances) unless explicitly non-transferable; correct events etc. |  |

| asset() | Returns the address of the underlying asset (an ERC-20 token) used by the vault. |  (Native Token, not ERC20) |

| totalAssets() | Returns total underlying assets managed by the vault (including yield) in asset units. Must reflect what is backing all shares. |  (Also considers isolated; could cause issues for integrations) |

| convertToShares(uint256 assets) | Given assets, compute how many shares one would get. Rounding: usually "floor". Must not depend on caller. |  (But depends on `msg.value`) |

| convertToAssets(uint256 shares) | Given shares, compute how many assets they are worth. Rounding: usually "floor". Must not depend on caller. |  (But depends on `msg.value`) |

| maxDeposit(address receiver) | Maximum assets that can be deposited. Must be $\geq$ deposit amount. Should obey vault's limits if any. |  (No stake-size limit in Monad) |

| previewDeposit(uint256 assets) | Simulate `deposit(assets, receiver)`: number of shares received. Should floor rounding. Must not revert. |  (But depends on `msg.value`) |

| deposit(uint256 assets, address receiver) | Transfers in assets, mints shares, emits Deposit event. Must fail if $\text{assets} > \text{maxDeposit}$. |  |

| maxMint(address receiver) | Maximum shares that can be minted. Must align with mint behavior. |  (No stake-size limit in Monad) |

| previewMint(uint256 shares) | Simulate `mint(shares, receiver)`: how many assets must be supplied. Rounding: usually "ceil". Must not revert. |  (But depends on `msg.value`) |

| mint(uint256 shares, address receiver) | Takes in assets, mints shares, ensures compliance with previewMint, emits Deposit. Reverts if $> \text{maxMint}$. |  |

| maxWithdraw(address owner) | Maximum assets owner can withdraw (considering shares, liquidity, fees). |  (Does not consider liquidity) |

| previewWithdraw(uint256 assets) | Simulate withdraw: shares required. Rounding: "ceil". Must not revert. Must be liquidity-agnostic. |  |

I-04 ERC4626 Compliance

CONTINUED

| withdraw(uint256 assets, address receiver, address owner) | Burns shares, transfers assets, emits Withdraw event, respects allowance. Must revert if > maxWithdraw(owner). |  |

| maxRedeem(address owner) | Maximum shares owner may redeem; equivalent to what redeem allows. |  (Liquidity not considered) |

| previewRedeem(uint256 shares) | Simulate redeem: assets returned. Rounding: "floor" after fees. Must not revert; liquidity-agnostic. |  |

| redeem(uint256 shares, address receiver, address owner) | Burns shares, transfers assets, emits Withdraw, respects allowance. Reverts if > maxRedeem(owner). |  |

| Rounding behavior across functions | Deposit/previewDeposit floors shares; mint/previewMint ceils assets; withdraw ceils shares; redeem floors assets. Previews must mirror. |  |

| No caller-dependence for view functions | Outputs of convert/previews must not vary with msg.sender. |  (Depend on `msg.value`) |

| Asset accounting invariants | totalAssets must track underlying managed; shares/asset conversions consistent; no leakage. |  |

| Fee behavior | If fees exist, previews/convert must reflect them; preview must never revert. |  |

| Signature/type compliance | Function signatures must match ERC-4626 exactly. |  |

— RECOMMENDATION

Not recorded.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Validators Handling

CATEGORY	Informational
LOCATION	—
REVIEW	Main Review

DESCRIPTION

Regarding Validators Handling What Lido does (reference point):

- Lido routes deposits round-robin to the next node operator with available capacity (not pro-rata).
- That capacity is set by governance (pre-submitted validator keys / limits).
- Rewards and penalties are socialized at the pool level.
- For unstake, which validators to exit is decided off-chain in coordination with withdrawal demand. Exits are scheduled centrally in the complete unstake step, not round-robin per user.

Now for Fastlane

If rewards/yield are only realized at epoch boundaries, we can mirror the simplicity and predictability:

****At the start of each epoch n:****

1. Realize yield for all validators.
2. Maintain one round-robin pointer:
 - `i` = next validator to stake to.

****During epoch n (after realization at t = 0):****

- Stake flow: delegate to validator `i`; mint according to **current `realAssets`** (epoch n)**. Mid-epoch changes don't affect the minting basis. `i++`.
- Unstake flow:
 - `requestUnstake` calculates the owed amount at the current ratio and records it.
 - Later, when Fastlane processes `completeUnstake`, it selects validators to exit according to weight / available balance and in a coordinated manner (not tied to a specific user), ensuring pool health and fairness.

Why this helps:

I-05 Validators Handling

CONTINUED

- Avoids the mis-accounting described here: <https://t.me/c/2763992473/2/69>.
- Keeps accounting deterministic (single source of truth per epoch).
- Likely lower gas than per-tx pro-rata bookkeeping across all validators.

Reference: [Lido Staking Vaults (stVaults) Technical Design and Architecture - HackMD]
(<https://hackmd.io/@lido/stVaults-design>)

— RECOMMENDATION

Not recorded.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.